mra.py 中带采样率编码的 Transformer 异常检测模块说明手册

2026-04-10

摘要

本文面向仓库当前版本的 `mra.py`，系统说明其中**带采样率编码的 Transformer 异常检测模块**及其直接相关的数据流、训练流和测试流。文档只聚焦 `SamplingAwareTransformerAD`、它的代码入口与训练入口、它与双专家门控插补器之间的耦合关系，以及训练集上学习得到的参数和统计量如何在测试阶段发挥作用。报告重点回答八个问题：第一，真正的代码入口在哪里；第二，参数更新的训练入口在哪里；第三，训练期额外遮挡、结构性缺失和补全序列之间是什么关系；第四，采样率编码、相位编码与变量编码如何共同构造 Transformer 输入；第五，检测器的数学原理与设计思路是什么；第六，单个 batch 内部张量如何逐层流动、形状如何变化；第七，各项损失分别更新哪些参数，尤其是 `detector_loss` 如何反向影响插补器；第八，训练集估计出的均值方差、采样率元数据、分数归一化参数与阈值如何作用于测试集。文中所有结论均以代码实现为依据，并辅以一次真实 batch 的形状检查与梯度探针结果。

关键词： 多采样率，Transformer 异常检测，采样率编码，相位编码，自监督重构，联合训练

目录

1 文档范围与核心结论 **4**
- 1.1 分析范围 4
- 1.2 先给结论 4

2 代码入口、训练入口与模块边界 **5**
- 2.1 入口总览 5
- 2.2 必须区分“代码入口”和“训练入口” 6
- 2.3 主调用链 6

3 数据、窗口与采样率元数据 **6**
- 3.1 原始数据与掩码 6
- 3.2 标准化器的含义 6
- 3.3 训练集中的三种采样率 7
- 3.4 相位编码的直观解释 7
- 3.5 窗口化后的真实形状 7

4 检测器上游：它究竟接收什么输入 **8**
- 4.1 三种掩码必须分清 8
- 4.2 输入窗口、种子窗口与完成窗口 8

4.3 双专家门控输出 8
4.4 为什么检测器吃的是 X_complete 9

5 SamplingAwareTransformerAD 的数学原理与设计思路 9
5.1 设计目标 9
5.2 逐变量局部表示 9
5.3 为什么先做逐变量编码，再做时间级 token 10
5.4 时间步 token 的构造 10
5.5 位置编码与 Transformer 编码器 11
5.6 重构头与残差输出 11

6 代码实现与张量形状逐层解析 11
6.1 上游到检测器的关键张量 11
6.2 检测器内部张量 12
6.3 把前向过程写成一串公式 12
6.4 一个重要实现细节：观测掩码既进入输入，又进入损失 13

7 训练流程、损失函数与参数更新机制 13
7.1 训练循环的真实执行顺序 13
7.2 五项损失的数学定义 13
7.3 检测器损失为什么是“完成窗口到完成窗口”的重构 14
7.4 训练期 holdout 对检测器目标的影响 14
7.5 梯度如何传播 15
7.6 各损失更新哪些参数 15

8 训练集学到的参数与统计量，在测试集中怎么发挥作用 15
8.1 要区分“可学习参数”和“训练集估计量” 15
8.2 训练期产物总表 15
8.3 这些量对测试集的具体作用链条 16
8.4 一个重要假设 16

9 测试阶段的数据流与异常分数形成机制 17
9.1 测试阶段不再有 holdout 17
9.2 检测器分数 17
9.3 专家分歧分数与门控熵 17
9.4 分布偏移分数 17
9.5 最终异常分数 18

10 默认配置下的实现规模 18
10.1 与检测器直接相关的默认超参数 18
10.2 检测器参数量分布 19

11 设计优点、隐含假设与局限 19
11.1 设计优点 19
11.2 隐含假设 19

11.3 局限与可改进点 20

12 结论 **20**

1 文档范围与核心结论

1.1 分析范围

本文只分析以下与采样率感知 Transformer 检测器直接相关的内容：

- `mra.py` 中 `SamplingAwareTransformerAD`、`FusionAnomalyModel`、`compute_losses`、`train_model`、`collect_window_statistics` 的实现；
- `mra.py` 中与检测器输入直接相关的上游部分，即 `PreImpute`、双专家门控融合和 $\boldsymbol{X}_{\text{complete}}$ 的构造方式；
- `utils/methods/windowing.py`、`utils/methods/data_loading.py`，以及 `graph_gcn_reconstruction.py` 中和数据尺度、窗口形状、标准化、采样率元数据推断相关的函数；
- 检测器输出如何转化为训练分数、测试分数与最终异常预测。

以下内容不是本文重点，因此只在必要耦合处简要提及：

- 图专家 `GraphGCNReconstructor` 的完整数学推导；
- 频域专家 `FrequencyOnlyReconstructor` 的完整频谱建模细节；
- EWAF、阈值函数与分类指标的统计学背景。

1.2 先给结论

结合代码、真实数据维度检查和单 batch 梯度验证，可以先给出九条核心结论：

- 带采样率编码的异常检测器定义在 `mra.py:343--455` 的 `SamplingAwareTransformerAD` 中，真正的前向入口在 `FusionAnomalyModel.forward` 里的 `self.detector(outputs["x_complete"], observed_mask, rate_id, stride)`。
- 当用户运行 `python mra.py` 时，真正发生参数更新的训练入口不是独立实验脚本，而是 `mra.py:548--621` 的 `train_model`；检测器与双专家插补器共享同一个 AdamW 优化器，属于**联合训练**。
- 检测器并不直接接收原始缺失窗口 $\boldsymbol{X}_{\text{input}}$，而是接收门控插补后的完成窗口 $\boldsymbol{X}_{\text{complete}}$。因此它学习的是“正常补全后窗口”的可重构性，而不是对原始缺失输入做端到端补全。
- 采样率感知不是单一编码，而是四种信息共同作用：变量编号嵌入、采样率类别嵌入、按步长取模得到的相位编码，以及显式观测掩码。
- 检测器的 token 不是“变量级 token”，而是“时间步级 token”。每个时间步先做逐变量特征编码，再把全部变量拼成一个时间 token 送入 Transformer。这大幅降低了注意力复杂度。
- `detector_loss` 的目标张量使用了 `detach()`，但输入张量 $\boldsymbol{X}_{\text{complete}}$ 没有被截断，因此 `detector_loss` 仍然会反向更新 GCN 专家、频域专家和门控网络；它只阻断了“把目标值当作可学习输出再回传”的那条分支。

- 检测器训练时监督位置不是“所有完成值”，而是**结构性观测位置**。这意味着原生缺失点不会直接参与检测器损失；训练期额外 holdout 点会以“插补后的完成值”身份参与检测器目标。
- 训练集不仅更新神经网络参数，还估计出一组非梯度统计量：标准化器均值方差、`rate_id`、`stride`、各分数源的训练均值与标准差，以及最终阈值。这些量在测试阶段都会继续发挥作用。
- 当前默认数据下，训练集和测试集均为 4100×18，窗口长度为 50，采样率分为三组：步长 $1, 3, 5$。检测器默认输入 token 维度为 324，Transformer 隐藏维度为 128，总参数量为 656,466。

2 代码入口、训练入口与模块边界

2.1 入口总览

位置	代码定位	作用
主程序入口	`mra.py` `:831--1098`	解析参数、读取训练/测试数据、估计标准化器、构建窗口、推断采样率元数据、训练模型、完成补全、计算分数、保存结果。
采样率元数据入口	`mra.py` `:139--150`	从训练集缺失率推断每个变量的采样步长 `stride` 与采样率类别 `rate_id`。
训练期额外遮挡	`mra.py` `:162--182`	在原本可观测的位置随机采样 holdout 点，构造自监督插补目标。
时域预插补	`mra.py` `:185--248`	通过双向填充产生 $\boldsymbol{X}_{\text{seed}}$，供上下游插补器使用。
双专家门控入口	`mra.py` `:270--340`	实例化图专家、频域专家与门控网络，生成 $\boldsymbol{X}_{\text{complete}}$。
检测器定义	`mra.py` `:343--455`	定义采样率感知的 Transformer 检测器 `SamplingAwareTransformerAD`。
联合模型封装	`mra.py` `:457--509`	用 `FusionAnomalyModel` 把插补器与检测器拼成完整前向链。
联合损失定义	`mra.py` `:512--545`	定义 `fusion/gcn/freq/detector/graph` 五项损失及总损失。
真实训练入口	`mra.py` `:548--621`	运行 `mra.py` 时实际发生参数更新的位置：前向、损失、反向传播、梯度裁剪、AdamW 更新都在这里完成。
补全推理入口	`mra.py` `:624--662`	用训练后的模型输出全序列补全结果、门控权重和邻接矩阵样本。
检测统计入口	`mra.py` `:665--722`	计算 `detector_score`、`disagreement_score` 和 `gate_entropy`。
窗口构造	`utils/methods/windowing.py` `:79--167`	生成训练插补窗口、训练评分窗口和测试评分窗口。
数据加载与掩码	`utils/methods/data_loading.py` `:80--97`	读取 CSV，并把 `NaN` 转成结构性缺失掩码。
标准化器	`graph_gcn_reconstruction.py` `:167--206`	只用观测值拟合均值方差，并把缺失位置映射为标准化后的零。

2.2 必须区分“代码入口”和“训练入口”

这份实现里，检测器的**代码入口**和**训练入口**不是一回事：

1. **代码入口**回答“检测器从哪里被调用”。核心调用语句是：

```
detector_reconstruction = self.detector(
outputs["x_complete"], observed_mask, rate_id, stride)
```

2. **训练入口**回答“检测器从哪里开始参与反向传播和参数更新”。核心更新链条是：

```
outputs = model(...)
losses = compute_losses(...)
losses["total_loss"].backward()
optimizer.step()
```

2.3 主调用链

按照真实执行顺序，相关调用链可写成

$$\begin{aligned}\texttt{main} &\to \texttt{train_model} \to \texttt{FusionAnomalyModel.forward}\\ &\to \texttt{TwoExpertGatedImputer.forward}\\ &\to \texttt{SamplingAwareTransformerAD.forward}\\ &\to \texttt{compute_losses} \to \texttt{AdamW.step}.\end{aligned} \tag{1}$$

这条链路说明：检测器不是独立模块，而是整个“插补 + 检测”联合系统中的后半段。

3 数据、窗口与采样率元数据

3.1 原始数据与掩码

`load_csv_glob_with_mask` 会读取 CSV，并把 `NaN` 转成结构性缺失掩码。对当前默认数据：

$$\boldsymbol{X}_{\text{raw}}^{\text{train}} \in \mathbb{R}^{4100\times 18}, \tag{2}$$

$$\boldsymbol{X}_{\text{raw}}^{\text{test}} \in \mathbb{R}^{4100\times 18}, \tag{3}$$

$$\boldsymbol{M}_s^{\text{train}}, \boldsymbol{M}_s^{\text{test}} \in \{0,1\}^{4100\times 18}. \tag{4}$$

其中 $M_{s,t,n}=1$ 表示第 t 个时间步、第 n 个变量在原始 CSV 中缺失。

3.2 标准化器的含义

`ObservedStandardScaler` 只用观测值估计每个变量的均值 μ_n 和标准差 σ_n，然后把缺失点先用 μ_n 填充，再进行标准化。因此标准化后的输入满足

$$X_{t,n} = \begin{cases} \dfrac{X_{t,n}^{\text{raw}} - \mu_n}{\sigma_n}, & M_{s,t,n}=0,\\ 0, & M_{s,t,n}=1. \end{cases} \tag{5}$$

这意味着数值上的零并不等于“真实值为零”，而是“缺失位置被观测均值标准化后恰好映射为零”。模型必须联合使用掩码来判断该值是否真实可见。

3.3 训练集中的三种采样率

`infer_rate_metadata` 根据训练集缺失率推断每个变量的观测率与采样步长：

$$r_n = 1 - \frac{1}{T_{\text{all}}} \sum_{t=1}^{T_{\text{all}}} M_{s,t,n}, \qquad s_n = \text{round}\left(\frac{1}{\max(r_n, 10^{-6})}\right). \tag{6}$$

对当前默认数据，训练集中的 18 个变量被划分为三组：

变量组	列索引	缺失率	观测率	推断步长 s_n	`rate_id`
高频组	1–6	0.0	1.0	1	0
中频组	7–12	0.6667	0.3333	3	1
低频组	13–18	0.8	0.2	5	2

因此默认运行时有

$$\boldsymbol{stride} = [1,1,1,1,1,1,3,3,3,3,3,3,5,5,5,5,5,5], \tag{7}$$

$$\boldsymbol{rate_id} = [0,0,0,0,0,0,1,1,1,1,1,1,2,2,2,2,2,2]. \tag{8}$$

3.4 相位编码的直观解释

检测器并不只知道“某个变量属于哪一类采样率”，还知道“当前时间步位于该采样周期的哪个相位”。代码中的相位定义为

$$\phi_{t,n} = \frac{\text{mod}(t, s_n)}{s_n}, \qquad \phi_{t,n} \in [0,1). \tag{9}$$

对于默认数据，不同采样组的相位序列分别近似为：

- 步长 1：$0, 0, 0, \ldots$；
- 步长 3：$0, \frac{1}{3}, \frac{2}{3}, 0, \frac{1}{3}, \frac{2}{3}, \ldots$；
- 步长 5：$0, 0.2, 0.4, 0.6, 0.8, 0, 0.2, \ldots$。

这使模型能区分“同样属于慢变量，但当前处于采样周期的哪个位置”。这在多采样率场景下很关键，因为慢变量在相邻时间步的语义并不等价。

3.5 窗口化后的真实形状

`mra.py` 里同时构建三类窗口：

- `build_windows`：用于训练插补和重建完整序列，支持前端补齐；
- `build_standard_windows`：用于训练分数统计；
- `build_prompt_test_windows`：用于测试分数统计，并生成固定标签。

默认 `seq_len=50`、`stride=1` 时，真实形状为

$$\boldsymbol{W}_{\text{imp}}^{\text{train}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{10}$$

$$\boldsymbol{W}_{\text{imp}}^{\text{test}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{11}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{train}} \in \mathbb{R}^{4001\times 50\times 18}, \tag{12}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{test}} \in \mathbb{R}^{4000\times 50\times 18}. \tag{13}$$

测试标签由 `build_prompt_test_windows` 生成，前 2000 个窗口标为正常，后 2000 个窗口标为异常。

4 检测器上游：它究竟接收什么输入

4.1 三种掩码必须分清

对一个训练 batch，记真实标准化窗口为

$$\boldsymbol{X}_{\text{true}} \in \mathbb{R}^{B\times T\times N}, \tag{14}$$

其中默认 $T = 50$、$N = 18$。代码中会同时出现三种掩码：

- 结构性缺失掩码 $\boldsymbol{M}_s$：原始数据本来就缺失的位置；
- 训练期额外遮挡掩码 $\boldsymbol{M}_h$：从可观测位置额外随机抽掉的位置；
- 模型输入掩码 $\boldsymbol{M}$：二者按元素取最大值后的结果。

写成公式即

$$\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h). \tag{15}$$

4.2 输入窗口、种子窗口与完成窗口

训练时先构造被遮挡后的模型输入

$$\boldsymbol{X}_{\text{input}} = \boldsymbol{X}_{\text{true}} \odot (1 - \boldsymbol{M}), \tag{16}$$

然后通过 `PreImpute.fill` 生成更平滑的种子窗口

$$\boldsymbol{X}_{\text{seed}} = \text{PreImpute}(\boldsymbol{X}_{\text{input}}, \boldsymbol{M}). \tag{17}$$

`PreImpute` 的作用并不是最终插补，而是先做一次双向时域补种：对每个变量沿时间轴做前向填充和后向填充，在两侧都有值时取均值，在只有单侧可用时取单侧值。这样可以避免图专家和频域专家直接面对大块零值。

4.3 双专家门控输出

设图专家和频域专家分别输出

$$\boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}} \in \mathbb{R}^{B\times T\times N}. \tag{18}$$

门控网络对每个 (b, t, n) 位置构造局部特征

$$\boldsymbol{g}_{b,t,n} = \left[x_{b,t,n}^{\text{seed}}, M_{b,t,n}, x_{b,t,n}^{\text{gcn}}, x_{b,t,n}^{\text{freq}}, x_{b,t,n}^{\text{gcn}} - x_{b,t,n}^{\text{seed}}, x_{b,t,n}^{\text{freq}} - x_{b,t,n}^{\text{seed}}, \boldsymbol{e}_{\text{rate},n}^{\text{gate}}\right], \tag{19}$$

其中 $\boldsymbol{e}_{\text{rate},n}^{\text{gate}} \in \mathbb{R}^8$ 是门控侧的采样率嵌入。

门控 logits 经 softmax 得到两位专家权重

$$\boldsymbol{\pi}_{b,t,n} = \mathrm{softmax}(\mathrm{MLP}(\boldsymbol{g}_{b,t,n})) \in \mathbb{R}^2, \tag{20}$$

于是融合插补结果为

$$x^{\mathrm{imp}}_{b,t,n} = \pi^{(0)}_{b,t,n} x^{\mathrm{gcn}}_{b,t,n} + \pi^{(1)}_{b,t,n} x^{\mathrm{freq}}_{b,t,n}. \tag{21}$$

最终完成窗口定义为

$$\boldsymbol{X}_{\mathrm{complete}} = \boldsymbol{M} \odot \boldsymbol{X}_{\mathrm{imp}} + (1 - \boldsymbol{M}) \odot \boldsymbol{X}_{\mathrm{input}}. \tag{22}$$

4.4 为什么检测器吃的是 `X_complete`

这是实现中的关键设计选择。检测器并不重复承担“从缺失到完整”的职责，而是接在插补器之后，学习“完成后的窗口是否符合训练期正常模式”。这有三个直接后果：

- 检测器面对的是一个**统一长度**、**统一数值域**、**缺失已补全**的时序窗口，建模难度更稳定；
- 检测器仍能通过 $\boldsymbol{O} = 1 - \boldsymbol{M}_s$ 知道哪些位置原本是观测点、哪些只是完成值，因此不会把“补出来的值”和“真实观测值”混为一谈；
- 插补器学得好坏会直接影响检测器输入分布，因此检测器损失会反过来约束插补器。

5 SamplingAwareTransformerAD 的数学原理与设计思路

5.1 设计目标

`SamplingAwareTransformerAD` 的核心目标不是普通窗口重构，而是把**多采样率信息显式注入 Transformer token**，使模型知道：

1. 当前值属于哪个变量；
2. 当前变量属于哪个采样率组；
3. 当前时间步处在该变量采样周期的哪个相位；
4. 当前值是原始观测还是插补完成值。

换句话说，它不是只用“数值本身”去判断异常，而是把“数值 + 采样制度 + 可见性”一起编码。

5.2 逐变量局部表示

对于窗口内第 t 个时间步、第 n 个变量，检测器首先构造四类局部信息：

- 完成值 $x^{\mathrm{complete}}_{b,t,n} \in \mathbb{R}$；
- 可见性 $o_{b,t,n} = 1 - M_{s,b,t,n} \in \{0, 1\}$；
- 变量嵌入 $\boldsymbol{e}^{\mathrm{var}}_n \in \mathbb{R}^8$；
- 采样率嵌入 $\boldsymbol{e}^{\mathrm{rate}}_n \in \mathbb{R}^8$；

- 相位嵌入 $\boldsymbol{e}_{t,n}^{\text{phase}} \in \mathbb{R}^{d_\phi}$。

其中相位标量由

$$\phi_{t,n} = \frac{\text{mod}(t, s_n)}{s_n} \tag{23}$$

给出，再经两层 MLP 映射为

$$\boldsymbol{e}_{t,n}^{\text{phase}} = f_\phi(\phi_{t,n}) \in \mathbb{R}^{d_\phi}, \qquad d_\phi = 8(\text{默认}) \ . \tag{24}$$

于是逐变量输入向量为

$$\boldsymbol{u}_{b,t,n} = \left[x_{b,t,n}^{\text{complete}}, o_{b,t,n}, \boldsymbol{e}_n^{\text{var}}, \boldsymbol{e}_n^{\text{rate}}, \boldsymbol{e}_{t,n}^{\text{phase}}\right] \in \mathbb{R}^{1+1+8+8+8} = \mathbb{R}^{26}. \tag{25}$$

随后经逐变量投影网络

$$\boldsymbol{h}_{b,t,n} = f_{\text{pv}}(\boldsymbol{u}_{b,t,n}) \in \mathbb{R}^{d_{\text{pv}}}, \qquad d_{\text{pv}} = 16(\text{默认}) \ . \tag{26}$$

5.3 为什么先做逐变量编码，再做时间级 token

这一步体现了实现的一个重要工程折中。若把每个“变量-时间”二元组都视为一个 Transformer token，则窗口长度会从 T 变成 $T \times N$。默认数据下：

$$T = 50, \qquad N = 18, \qquad T \times N = 900. \tag{27}$$

而当前实现采用“每个时间步一个 token”的方式，只保留 50 个 token，再把该时间步上的全部变量信息拼到 token 特征里。这样做有两个优点：

- 注意力复杂度从大致 $O((TN)^2)$ 降到 $O(T^2)$；
- 仍然保留变量级别的采样率、相位和可见性信息，因为这些信息已经在 $\boldsymbol{h}_{b,t,n}$ 中编码过。

代价是：Transformer 编码器内部不再显式建模变量间 token 级注意力，而是把变量间交互留给拼接表示与上游插补器去承担。

5.4 时间步 token 的构造

对每个时间步 t，代码把三部分拼成一个大向量：

$$\boldsymbol{z}_{b,t} = \left[\boldsymbol{X}_{b,t,:}^{\text{complete}}, \boldsymbol{O}_{b,t,:}, \text{vec}(\boldsymbol{H}_{b,t,:,:})\right]. \tag{28}$$

其中

$$\boldsymbol{X}_{b,t,:}^{\text{complete}} \in \mathbb{R}^N, \qquad \boldsymbol{O}_{b,t,:} \in \mathbb{R}^N, \qquad \text{vec}(\boldsymbol{H}_{b,t,:,:}) \in \mathbb{R}^{Nd_{\text{pv}}}. \tag{29}$$

因此 token 输入维度为

$$d_{\text{token-in}} = N + N + Nd_{\text{pv}} = N(2 + d_{\text{pv}}). \tag{30}$$

对默认配置 $N = 18$、$d_{\text{pv}} = 16$：

$$d_{\text{token-in}} = 18 \times 18 = 324. \tag{31}$$

随后线性投影到 Transformer 隐藏维度

$$\boldsymbol{q}_{b,t} = \boldsymbol{W}_{\text{tok}}\boldsymbol{z}_{b,t} + \boldsymbol{b}_{\text{tok}} \in \mathbb{R}^{d_{\text{model}}}, \qquad d_{\text{model}} = 128(\text{默认}) \ . \tag{32}$$

5.5 位置编码与 Transformer 编码器

代码对每个时间步加入固定的正弦位置编码

$$\boldsymbol{q}'_{b,t} = \boldsymbol{q}_{b,t} + \boldsymbol{p}_t, \tag{33}$$

其中 $\boldsymbol{p}_t$ 由 `SinusoidalPositionalEncoding` 给出。

之后送入 `nn.TransformerEncoder`：

$$\boldsymbol{H}_{b,:,:} = \mathrm{TransformerEncoder}\left(\boldsymbol{Q}'_{b,:,:}\right) \in \mathbb{R}^{T\times d_{\mathrm{model}}}. \tag{34}$$

默认配置下，编码器采用：

- 3 层 `TransformerEncoderLayer`；
- 4 个注意力头，因此每头维度为 $128/4 = 32$；
- 前馈层宽度 $4d_{\mathrm{model}} = 512$；
- `GELU` 激活与 `dropout=0.1`；
- `batch_first=True`；
- 不使用因果 mask，因此是**双向窗口编码**，适合离线窗口异常检测，而不是在线自回归预测。

5.6 重构头与残差输出

编码器输出并不直接作为异常分数，而是先经重构头预测一个残差修正量

$$\Delta\boldsymbol{X}_{\mathrm{det}} = f_{\mathrm{head}}(\boldsymbol{H}) \in \mathbb{R}^{B\times T\times N}. \tag{35}$$

最终检测器输出为

$$\widehat{\boldsymbol{X}}_{\mathrm{det}} = \boldsymbol{X}_{\mathrm{complete}} + \Delta\boldsymbol{X}_{\mathrm{det}}. \tag{36}$$

这种残差式设计有两个工程优点：

- 若输入窗口已经接近正常模式，最容易学习的映射就是 $\Delta\boldsymbol{X}_{\mathrm{det}} \approx 0$，训练更稳定；
- 检测器更像是在“校正完成序列”，而不是从零开始重新生成整段窗口。

6 代码实现与张量形状逐层解析

6.1 上游到检测器的关键张量

对任意 batch 大小 B，默认窗口长度 $T = 50$、变量数 $N = 18$。上游关键张量如下：

张量	含义	一般形状	默认实例
$\boldsymbol{X}_{\mathrm{true}}$	真实标准化窗口	$B\times T\times N$	$B\times 50\times 18$
$\boldsymbol{M}_s$	结构性缺失掩码	$B\times T\times N$	$B\times 50\times 18$
$\boldsymbol{M}_h$	训练期额外遮挡掩码	$B\times T\times N$	$B\times 50\times 18$
$\boldsymbol{M}$	模型输入掩码	$B\times T\times N$	$B\times 50\times 18$

张量	含义	一般形状	默认实例
$\boldsymbol{X}_{\text{input}}$	零遮挡后的输入窗口	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{X}_{\text{seed}}$	`PreImpute` 种子窗口	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{X}_{\text{gcn}}$	图专家输出	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{X}_{\text{freq}}$	频域专家输出	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{\Pi}$	两专家门控权重	$B \times T \times N \times 2$	$B \times 50 \times 18 \times 2$
$\boldsymbol{X}_{\text{imp}}$	门控融合后的插补结果	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{X}_{\text{complete}}$	完成窗口	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{O}$	结构性观测掩码 $= 1 - \boldsymbol{M}_s$	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{rate_id}$	变量所属采样率类别	N	18
$\boldsymbol{stride}$	变量推断采样步长	N	18

6.2 检测器内部张量

根据一次真实前向检查，`SamplingAwareTransformerAD` 内部主要张量形状如下：

张量	来源	一般形状	默认实例
$\boldsymbol{E}_{\text{var}}$	变量嵌入表查表结果	$N \times 8$	18×8
$\boldsymbol{E}_{\text{rate}}$	采样率嵌入表查表结果	$N \times 8$	18×8
$\boldsymbol{\Phi}$	标量相位	$T \times N \times 1$	$50 \times 18 \times 1$
$\boldsymbol{E}_{\text{phase}}$	相位 MLP 输出	$T \times N \times 8$	$50 \times 18 \times 8$
$\boldsymbol{U}$	逐变量输入拼接	$B \times T \times N \times 26$	$B \times 50 \times 18 \times 26$
$\boldsymbol{H}_{\text{pv}}$	逐变量投影结果	$B \times T \times N \times 16$	$B \times 50 \times 18 \times 16$
$\boldsymbol{Z}$	时间步 token 输入	$B \times T \times 324$	$B \times 50 \times 324$
$\boldsymbol{Q}$	token 投影结果	$B \times T \times 128$	$B \times 50 \times 128$
$\boldsymbol{P}$	正弦位置编码	$1 \times T \times 128$	$1 \times 50 \times 128$
$\boldsymbol{H}$	Transformer 编码结果	$B \times T \times 128$	$B \times 50 \times 128$
$\Delta\boldsymbol{X}_{\text{det}}$	重构头残差输出	$B \times T \times N$	$B \times 50 \times 18$
$\widehat{\boldsymbol{X}}_{\text{det}}$	检测器重构结果	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{E}_{\text{det}}$	检测器绝对误差	$B \times T \times N$	$B \times 50 \times 18$

6.3 把前向过程写成一串公式

将代码中的 `forward` 逻辑合并，可写成

$$\boldsymbol{U} = \left[\boldsymbol{X}_{\text{complete}} \uparrow, \boldsymbol{O} \uparrow, \boldsymbol{E}_{\text{var}}^{\uparrow}, \boldsymbol{E}_{\text{rate}}^{\uparrow}, \boldsymbol{E}_{\text{phase}}^{\uparrow}\right], \tag{37}$$

其中 $\uparrow$ 表示通过 `unsqueeze/expand` 广播到 $B \times T \times N$ 级别。

然后

$$\boldsymbol{H}_{\text{pv}} = f_{\text{pv}}(\boldsymbol{U}), \tag{38}$$

$$\boldsymbol{Z} = \left[\boldsymbol{X}_{\text{complete}}, \boldsymbol{O}, \text{vec}(\boldsymbol{H}_{\text{pv}})\right], \tag{39}$$

$$\boldsymbol{Q} = \boldsymbol{W}_{\text{tok}}\boldsymbol{Z} + \boldsymbol{b}_{\text{tok}}, \tag{40}$$

$$\boldsymbol{H} = \mathrm{TransformerEncoder}(\boldsymbol{Q} + \boldsymbol{P}), \tag{41}$$

$$\Delta \boldsymbol{X}_{\mathrm{det}} = f_{\mathrm{head}}(\boldsymbol{H}), \tag{42}$$

$$\widehat{\boldsymbol{X}}_{\mathrm{det}} = \boldsymbol{X}_{\mathrm{complete}} + \Delta \boldsymbol{X}_{\mathrm{det}}. \tag{43}$$

6.4 一个重要实现细节：观测掩码既进入输入，又进入损失

代码同时做了两件事情：

1. 把 $\boldsymbol{O} = 1 - \boldsymbol{M}_s$ 作为显式输入特征拼进 token；
2. 在损失和异常分数里，只对 $\boldsymbol{O}$ 指示的结构性观测位置计分。

这使检测器既能知道“这个值是不是原始观测”，又不会被原生缺失位置的伪目标污染。

7 训练流程、损失函数与参数更新机制

7.1 训练循环的真实执行顺序

`train_model` 的每个 batch 执行步骤如下：

1. 从 `DataLoader` 读入 $\boldsymbol{X}_{\mathrm{true}}$ 与 $\boldsymbol{M}_s$；
2. 用 `sample_holdout_mask` 采样 $\boldsymbol{M}_h$，默认比例为 0.15；
3. 计算 $\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h)$，并据此得到 $\boldsymbol{X}_{\mathrm{input}}$；
4. 调用 `model(...)`，依次完成预插补、双专家融合和检测器重构；
5. 调用 `compute_losses(...)` 计算五项损失；
6. `optimizer.zero_grad()`；
7. `losses["total_loss"].backward()`；
8. `clip_grad_norm_(max_norm=1.0)`；
9. `optimizer.step()`。

优化器是

$$\texttt{optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)}. \tag{44}$$

7.2 五项损失的数学定义

`masked_mse` 的通用形式是

$$\mathrm{MSE}_{\Omega}(\widehat{\boldsymbol{X}}, \boldsymbol{X}) = \frac{\sum_{b,t,n} \Omega_{b,t,n} \left(\widehat{X}_{b,t,n} - X_{b,t,n}\right)^2}{\sum_{b,t,n} \Omega_{b,t,n} + \varepsilon}. \tag{45}$$

于是五项损失分别为：

$$\mathcal{L}_{\mathrm{fusion}} = \mathrm{MSE}_{\boldsymbol{M}_h}(\boldsymbol{X}_{\mathrm{imp}}, \boldsymbol{X}_{\mathrm{true}}), \tag{46}$$

$$\mathcal{L}_{\text{gcn}} = \text{MSE}_{\boldsymbol{M}_h}(\boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{true}}), \tag{47}$$

$$\mathcal{L}_{\text{freq}} = \text{MSE}_{\boldsymbol{M}_h}(\boldsymbol{X}_{\text{freq}}, \boldsymbol{X}_{\text{true}}), \tag{48}$$

$$\mathcal{L}_{\text{graph}} = \text{mean}(\text{diag}(\boldsymbol{A}) - \delta)^2, \tag{49}$$

$$\mathcal{L}_{\text{det}} = \text{MSE}_{\boldsymbol{O}}\left(\widehat{\boldsymbol{X}}_{\text{det}}, \text{sg}(\boldsymbol{X}_{\text{complete}})\right), \tag{50}$$

其中 $\boldsymbol{A}$ 是图专家产生的邻接矩阵，$\delta = \texttt{diag_target} = 0.25$，$\text{sg}(\cdot)$ 表示停止梯度。

总损失为

$$\mathcal{L}_{\text{total}} = 1.0\,\mathcal{L}_{\text{fusion}} + 0.4\,\mathcal{L}_{\text{gcn}} + 0.4\,\mathcal{L}_{\text{freq}} + 0.5\,\mathcal{L}_{\text{det}} + 0.1\,\mathcal{L}_{\text{graph}}. \tag{51}$$

7.3 检测器损失为什么是“完成窗口到完成窗口”的重构

检测器损失里最容易误读的地方是

$$\texttt{detector_target = outputs["x_complete"].detach()}. \tag{52}$$

这意味着检测器不是去逼近原始真值 $\boldsymbol{X}_{\text{true}}$，而是去逼近当前插补器输出的完成窗口 $\boldsymbol{X}_{\text{complete}}$。这样设计的理由是：

- 检测器学习的是**正常完成序列的可重构性**；
- 插补器负责恢复缺失，检测器负责度量“完成后序列是否偏离正常模式”；
- 当检测器无法稳定重构完成窗口时，该窗口更可能对应异常模式。

7.4 训练期 holdout 对检测器目标的影响

这部分尤其值得写清楚。定义结构性观测集合

$$\Omega_s = \{(b,t,n) \mid M_{s,b,t,n} = 0\}, \tag{53}$$

以及训练期额外遮挡集合

$$\Omega_h = \{(b,t,n) \mid M_{h,b,t,n} = 1\}. \tag{54}$$

由于

$$\boldsymbol{X}_{\text{complete}} = \boldsymbol{M} \odot \boldsymbol{X}_{\text{imp}} + (1 - \boldsymbol{M}) \odot \boldsymbol{X}_{\text{input}}, \tag{55}$$

所以对训练样本中的三类位置，有

$$X_{b,t,n}^{\text{complete}} = \begin{cases} X_{b,t,n}^{\text{true}}, & (b,t,n) \in \Omega_s \setminus \Omega_h, \\ X_{b,t,n}^{\text{imp}}, & (b,t,n) \in \Omega_h, \\ X_{b,t,n}^{\text{imp}}, & M_{s,b,t,n} = 1. \end{cases} \tag{56}$$

而检测器损失只在 Ω_s 上计算，因为掩码是 $\boldsymbol{O} = 1 - \boldsymbol{M}_s$。于是：

- 对**普通观测点**，检测器目标是真实观测值；
- 对 **holdout 点**，检测器目标是插补器给出的完成值，而不是真值；
- 对**原生缺失点**，检测器不直接受监督。

这说明检测器训练的对象是“插补器定义下的完成序列流形”，而不是单纯的原始观测流形。

7.5 梯度如何传播

对检测器损失

$$\mathcal{L}_{\text{det}} = \frac{1}{|\Omega_s|} \sum_{(b,t,n)\in\Omega_s} \Big(D_{\theta_d}(\boldsymbol{X}_{\text{complete}})_{b,t,n} - \text{sg}(X^{\text{complete}}_{b,t,n}) \Big)^2, \tag{57}$$

其对 $\boldsymbol{X}_{\text{complete}}$ 的梯度只有输入支路，没有目标支路：

$$\frac{\partial \mathcal{L}_{\text{det}}}{\partial \boldsymbol{X}_{\text{complete}}} = \frac{2}{|\Omega_s|} \boldsymbol{O} \odot (D_{\theta_d}(\boldsymbol{X}_{\text{complete}}) - \text{sg}(\boldsymbol{X}_{\text{complete}})) \cdot \frac{\partial D_{\theta_d}(\boldsymbol{X}_{\text{complete}})}{\partial \boldsymbol{X}_{\text{complete}}}. \tag{58}$$

因此：

- 检测器参数 θ_d 会被更新；
- 由于 $\boldsymbol{X}_{\text{complete}}$ 由门控插补器产生，插补器参数也会被更新；
- 但目标侧不会把“把 $\boldsymbol{X}_{\text{complete}}$ 自身拉向某个方向”的梯度回传到插补器。

7.6 各损失更新哪些参数

结合代码依赖关系和一次真实 batch 的梯度探针，可把参数更新关系总结为：

损失项	GCN 专家	频域专家	门控网络	检测器
$\mathcal{L}_{\text{fusion}}$	是	是	是	否
$\mathcal{L}_{\text{gcn}}$	是	否	否	否
$\mathcal{L}_{\text{freq}}$	否	是	否	否
$\mathcal{L}_{\text{det}}$	是	是	是	是
$\mathcal{L}_{\text{graph}}$	是（主要是图学习器）	否	否	否
$\mathcal{L}_{\text{total}}$	是	是	是	是

真实梯度探针还给出了一个很重要的定性事实：$\mathcal{L}_{\text{det}}$ 对上游插补器的梯度范数明显小于对检测器自身的梯度范数，但**确实非零**。也就是说，检测器既在学习自己的正常模式重构能力，也在温和地反向塑造插补器输出的分布。

8 训练集学到的参数与统计量，在测试集中怎么发挥作用

8.1 要区分“可学习参数”和“训练集估计量”

运行 `mra.py` 后，测试阶段会用到两类东西：

1. 通过反向传播得到的**可学习参数**；
2. 仅在训练集上估计、但不会反向传播的**统计量或元数据**。

二者都对测试结果有影响，但机制不同。

8.2 训练期产物总表

训练期产物	来源	是否梯度学习	测试阶段作用
标准化器 μ_n, σ_n	`ObservedStandardScaler.fit`	否	用训练集观测值的均值方差统一标准化训练集和测试集，并在补全导出时做逆标准化。
$\boldsymbol{rate_id}$	`infer_rate_metadata`	否	作为检测器和门控的采样率类别嵌入索引。
$\boldsymbol{stride}$	`infer_rate_metadata`	否	用于检测器的相位计算 $\phi_{t,n}$。
插补器参数 θ_{imp}	AdamW + 反向传播	是	决定测试集上的 $\boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}}, \boldsymbol{X}_{\text{imp}}, \boldsymbol{X}_{\text{complete}}$。
检测器参数 θ_d	AdamW + 反向传播	是	决定测试集上的 $\widehat{\boldsymbol{X}}_{\text{det}}$ 与 `detector_score`。
训练分数均值/方差	`normalize_scores`	否	用训练集分数分布把训练分数和测试分数统一映射到 z-score 尺度。
训练阈值 τ	`choose_threshold`	否	用平滑后的训练分数确定最终测试判定阈值。

8.3 这些量对测试集的具体作用链条

测试阶段的作用链条可以概括为：

$$\text{训练集估计的}(\mu, \sigma, \boldsymbol{rate_id}, \boldsymbol{stride}, \theta_{\text{imp}}, \theta_d, \tau) \Longrightarrow \text{测试窗口补全与异常评分}. \tag{59}$$

更细地说：

- 若训练集拟合出的 μ_n, σ_n 改变，则测试值的标准化坐标会改变，进而影响插补器和检测器的全部输入分布；
- 若 $\boldsymbol{rate_id}$ 或 $\boldsymbol{stride}$ 改变，则相同测试窗口会得到不同的 rate embedding 和 phase embedding；
- 若 θ_{imp} 改变，则测试集 $\boldsymbol{X}_{\text{complete}}$ 会改变，
  进而同时改变 `detector_score`、`disagreement_score` 和 `shift_score`；
- 若 θ_d 改变，则在相同 $\boldsymbol{X}_{\text{complete}}$ 上得到的检测器重构误差会改变；
- 若训练分数均值方差改变，则测试分数归一化后的量纲会改变；
- 若阈值 τ 改变，则最终二值化预测会改变。

8.4 一个重要假设

`rate_id` 和 `stride` 完全由**训练集缺失率**推断而来，测试阶段直接复用。它隐含了一个假设：训练集和测试集的采样制度一致，至少近似一致。若测试集的缺失模式与训练集显著不同，这两个元数据就可能失真，进而影响检测器的相位编码和类别嵌入。

9 测试阶段的数据流与异常分数形成机制

9.1 测试阶段不再有 holdout

在 `reconstruct_full_sequence` 与 `collect_window_statistics` 中，模型调用方式是

$$\texttt{model(x_input, structural_missing_mask, structural_missing_mask, rate_id, stride)}. \tag{60}$$

也就是说，测试阶段有

$$\boldsymbol{M} = \boldsymbol{M}_s, \tag{61}$$

不再注入训练期额外遮挡。因此：

- 结构性观测点上的 $\boldsymbol{X}_{\text{complete}}$ 直接等于真实标准化值；
- 结构性缺失点上的 $\boldsymbol{X}_{\text{complete}}$ 来自门控插补结果；
- 检测器分数只在结构性观测点上计算。

9.2 检测器分数

`collect_window_statistics` 中的检测器分数定义为

$$s_b^{\text{det}} = \frac{\sum_{t,n} O_{b,t,n} \left(\widehat{X}_{b,t,n}^{\text{det}} - X_{b,t,n}^{\text{complete}} \right)^2}{\sum_{t,n} O_{b,t,n} + \varepsilon}. \tag{62}$$

这就是典型的窗口级重构误差。若测试窗口的时序模式符合训练期正常完成序列流形，则该分数应较小；若不符合，则分数增大。

9.3 专家分歧分数与门控熵

代码还计算了两个与检测器互补的量：

$$s_b^{\text{gap}} = \frac{1}{TN} \sum_{t,n} \left| X_{b,t,n}^{\text{gcn}} - X_{b,t,n}^{\text{freq}} \right|, \tag{63}$$

$$s_b^{\text{entropy}} = -\frac{1}{TN} \sum_{t,n} \sum_{k=1}^{2} \pi_{b,t,n}^{(k)} \log \pi_{b,t,n}^{(k)}. \tag{64}$$

其中：

- `disagreement_score` 进入最终异常分数；
- `gate_entropy` 只被保存到 CSV，用于诊断门控不确定性，但**不参与最终分数加权**。

9.4 分布偏移分数

`shift_score` 来源于完成窗口的统计特征偏移。代码先从完成窗口中提取以下特征：

- 列块 1–6、7–12、13–18、1–18 的均值与标准差；
- 时间一阶差分绝对值的平均；
- 前几个 FFT 频点幅值的平均。

然后用训练完成窗口的均值方差做 z-score 归一化，最后定义

$$s_b^{\text{shift}} = \sqrt{\frac{1}{d_f}\sum_{i=1}^{d_f} z_{b,i}^2}. \tag{65}$$

虽然这一步不在 Transformer 内部，但它会与检测器分数共同组成最终异常分数，因此属于测试阶段的下游关键链条。

9.5 最终异常分数

`combine_scores` 先对三类分数源分别用训练集均值方差归一化：

$$\widetilde{s} = \frac{s - \mu_{\text{train}}}{\sigma_{\text{train}}}. \tag{66}$$

然后把它们加权组合为原始最终分数：

$$s^{\text{raw}} = \left|\widetilde{s}^{\text{det}}\right| + \lambda_{\text{gap}}\widetilde{s}^{\text{gap}} + \lambda_{\text{shift}}\widetilde{s}^{\text{shift}}, \tag{67}$$

其中默认

$$\lambda_{\text{gap}} = 0.25, \qquad \lambda_{\text{shift}} = 1.0. \tag{68}$$

随后再做 EWAF 平滑，并用训练集分数确定阈值

$$\tau = \max\left(\mu_{\text{train}} + k\sigma_{\text{train}},\ q_{0.95}\right), \tag{69}$$

其中默认 $k = 2.0$，$q_{0.95}$ 为训练平滑分数的 0.95 分位数。

最终：

- 若测试分数 $\geq \tau$，先判为异常；
- 再通过最短持续长度过滤短片段，默认最短异常长度为 50。

10 默认配置下的实现规模

10.1 与检测器直接相关的默认超参数

参数	默认值	作用
`seq_len`	50	窗口长度
`detector_d_model`	128	Transformer 隐藏维度
`detector_heads`	4	多头注意力头数
`detector_layers`	3	编码器层数
`dropout`	0.1	编码器 dropout
`phase_dim`	8	相位嵌入维度
`variable_embed_dim`	8	变量嵌入维度
`rate_embed_dim`	8	检测器侧采样率嵌入维度
`per_variable_dim`	16	逐变量投影维度
`num_rates`	3	采样率类别数
`num_nodes`	18	变量数

10.2 检测器参数量分布

一次实际参数统计结果如下：

子模块	参数量	备注
变量嵌入	144	18×8
采样率嵌入	24	3×8
相位 MLP	88	两层小网络
逐变量投影	704	$26 \to 16 \to 16$
token 投影	41,600	$324 \to 128$
Transformer 编码器	594,816	检测器主体
重构头	19,090	$128 \to 128 \to 18$
检测器总计	656,466	占整模参数的大部分

整个 `FusionAnomalyModel` 的总参数量为 693,260，其中检测器约占 94.7%。这说明在当前实现里，Transformer 检测器是参数规模上的主体，而双专家插补器更像一个高质量输入构造器。

11 设计优点、隐含假设与局限

11.1 设计优点

从实现角度看，这个检测器有五个明显优点：

- **显式多采样率建模**：不是指望自注意力自己“猜出”慢变量和快变量，而是直接提供 `rate_id` 和 `stride`。
- **相位感知**：同样的慢变量，在采样周期的不同相位下拥有不同表示。
- **观测可见性显式输入**：模型知道哪些值来自原始观测、哪些值只是完成结果。
- **复杂度可控**：采用时间步级 token，而不是变量级 token，适合当前 $T = 50, N = 18$ 的窗口建模。
- **联合训练**：检测器不会孤立地学习，而是会反向约束插补器输出的正常性。

11.2 隐含假设

它也依赖若干不写在注释里、但代码实际上假设成立的前提：

- 缺失模式可以近似反映真实采样制度，因此可由训练集缺失率反推出 `stride`；
- 训练集和测试集采样制度大体一致；
- 正常窗口在完成后应当具有更高可重构性，异常窗口则更难被重构；
- 结构性缺失点不宜直接作为检测器损失监督位置。

11.3 局限与可改进点

从代码实现本身出发，可以看到四个局限：

- 相位 $\phi_{t,n}$ 由全局时间索引取模得到，表达的是**理想周期位置**，并非真实“距上次观测已过去多久”的 age 特征。
- `rate_id` 和 `stride` 只从训练集估计一次，若测试采样模式漂移，这两个编码会失配。
- 训练期 holdout 点在检测器损失里使用的是插补器完成值而非真实值，这会让检测器更偏向“重构插补器流形”，而不是严格重构真值流形。
- `gate_entropy` 被保存但未进入最终分数，说明当前实现还留有可进一步利用的门控不确定性信息。

12 结论

若只用一句话概括这套实现：`mra.py` 中的 `SamplingAwareTransformerAD` 并不是一个孤立的 Transformer 重构器，而是建立在门控插补结果之上的**多采样率感知窗口级异常检测器**。它通过变量嵌入、采样率嵌入、相位嵌入和观测掩码，把多采样率过程的结构信息显式送入时间步 token；再通过联合训练，使检测器损失不仅更新自身，也反向塑造插补器输出的完成序列分布。

对实际使用者来说，最重要的三点是：第一，真正的训练入口在 `mra.py` 的 `train_model`；第二，测试阶段发挥作用的不只是模型权重，还有训练集估计的 μ/σ、`rate_id`、`stride`、训练分数归一化参数和阈值；第三，异常分数并非单独来自 Transformer，而是由检测器误差、双专家分歧和完成窗口分布偏移共同组成。